

Chapter 7

More CSS Techniques

INT150 Web Technologies (2/2025)

Sanit Sirisawatvatana

Topics

- Styling forms
- Style properties for table
- CSS reset and normalizer
- Image replacement techniques
- CSS feature detection

Styling Forms

- By default, web forms can appear somewhat inconsistent and unpolished without styling.
- To achieve a more professional and cohesive design, it is advisable to apply custom CSS styles.
- Researches show that forms are much easier and faster to use when the labels and inputs are lined up nicely.
- There aren't any special CSS properties for styling forms; just use the standard color, background, font, border, margin and padding properties.
- Flexbox is a good tool for making the form adapt to available viewport space.

Styling Forms

Custom Sneaker Order Form

- Name:
- Email:
- Telephone:
- Tell us about yourself:
- Size: Sizes reflect standard US men's sizes
- Sneaker Color:
 - Red
 - Blue
 - Black
 - Silver
- Add-on Features:
 - Feature
 - ☐ Sparkley laces
 - ☒ Metallic logo
 - ☐ Light-up heels
 - ☐ MP3-enabled
-

Wide viewport

Custom Sneaker Order Form

Name:

Email:

Telephone:

Tell us about yourself:

Size: Sizes reflect standard US men's sizes

Sneaker Color: ☐ Red ☐ Blue ☐ Black ☐ Silver

Add-on Features:

- ☐ Sparkley laces
- ☒ Metallic logo
- ☐ Light-up heels
- ☐ MP3-enabled

Narrow viewport

Custom Sneaker Order Form

Name:

Email:

Telephone:

Tell us about yourself:

Size: Sizes reflect standard US men's sizes

Sneaker Color: ☐ Red ☐ Blue ☐ Black ☐ Silver

Add-on Features:

- ☐ Sparkley laces
- ☒ Metallic logo
- ☐ Light-up heels
- ☐ MP3-enabled

No style

CSS styles

Styling Forms

The following is a brief overview of the customization options for each form control type:

- **Text inputs**
 - Change the appearance of the box with box properties.
 - Style the text inside the box with color and various font properties.
- **Textarea**
 - Change font and line height of text field.
 - Prevent resizing with "resize: none ;".
- **Buttons (submit, reset, button)**
 - Use box and font properties.
 - By default, buttons are set to the border-box sizing. Most browsers also add a bit of padding by default, which can be overridden by your own padding value.
 - Apply style to the text that appear on the buttons.

Styling Forms

- Radio and checkbox buttons
 - Best practice is to leave them alone (or substitute your own JavaScript custom buttons).
- Drop-down and select menus (select)
 - Specify the width and height.
 - Best practice is to leave the option formatting to the browser.
 - Some allow you to style the option text.
- Fieldsets and legends
 - Apply box properties to fieldset.
 - The legend element is difficult to style (try styling a span inside the legend element instead).

Styling Tables

- To add space within a cell, apply the `padding` property to `td` or `th`.
- To add space between cells, use the `border-collapse` and `border-spacing` properties
- By default, borders are `separated`, and a border is drawn on all four sides of each cell.
- The `border-spacing` property specifies the space between cell borders.
- The values for `border-spacing` are two length measurements: `horizontal-length` and `vertical-length`. Providing one value will be used both horizontally and vertically.

border-collapse and border-spacing

border-collapse

Values: separate | collapse

Default: sperate

Applies to: table and inline-table elements

Inherits: yes

border-spacing (For separated border)

Values: horizontal-length vertical-length

Default: 0

Applies to: table and inline-table elements

Inherits: yes

Styling Tables

1	2
3	4

```
table {  
    border-collapse: separate ;  
}  
th, td {  
    border: 1px solid black ;  
}
```

1	2
3	4

```
table {  
    border-collapse: collapse ;  
}  
th, td {  
    border: 1px solid black ;  
}
```

5px

1	2
3	4

```
table {  
    border-collapse: separate ;  
    border-spacing: 5px 20px ;  
}  
th, td {  
    border: 1px solid black ;  
}
```

Collapsed Borders

- In the collapse border model
 - The borders of adjacent borders "collapse"
 - Only one of the borders is visible and the space is removed.
- The advantage to using the collapsed table-border model is that you can style the borders for `tr`, `col`, `rowgroup`, and `colgroup` elements (the separated model can't set properties).
- Strategic use of horizontal and vertical borders improves the readability of complicated table, making the collapsed model an attractive choice.

empty-cells

empty-cells

Values: [show](#) | hide

Default: show

Applies to: table cell elements

Inherits: yes

- For tables with [separated borders](#), you can specify whether empty cells to display their backgrounds and borders by using [empty-cells](#) property.
- For a cell to be "empty", it may not contain any text, images, or non-breaking spaces. It may contain carriage returns and space characters

empty-cells

1	2	
---	---	--

	3	4
--	---	---

1	2
---	---

3	4
---	---

```
table {  
    border-collapse: separate ;  
    border-spacing: 5px 20px ;  
}  
th, td {  
    border: 1px solid black ;  
    empty-cells: show ;  
}
```

```
table {  
    border-collapse: separate ;  
    border-spacing: 5px 20px ;  
}  
th, td {  
    border: 1px solid black ;  
    empty-cells: hide ;  
}
```

caption-side and table-layout

caption-side

Values: [top](#) | bottom

Default: top

Applies to: table and inline-table elements

Inherits: yes

- Specifies on which side the caption element displays.

table-layout

Values: [auto](#) | fixed

Default: auto

Applies to: table and inline-table elements

Inherits: yes

- Calculates the width of the table based on width values([fixed](#)) or the minimum width required for the content of the table([auto](#)).

caption-side

```
<table>
  <caption>Header</caption>
  <tr><td>1</td><td>2</td><td> </td></tr>
  <tr><td> </td><td>3</td><td>4</td></tr>
</table>
```

Header

1	2	
---	---	--

	3	4
--	---	---

1	2	
---	---	--

	3	4
--	---	---

Header

```
table {
  border-collapse: separate ;
  border-spacing: 5px 20px ;
}
th, td {
  border: 1px solid black ;
}
```

```
table {
  border-collapse: separate ;
  border-spacing: 5px 20px ;
  caption-side: bottom ;
}
th, td {
  border: 1px solid black ;
}
```

table-layout

```
<table>
  <tr>
    <td>Short</td>
    <td>Very very long content</td>
  </tr>
  <tr>
    <td>Medium</td>
    <td>Content</td>
  </tr>
</table>
```

auto

Short	Very very long content
Medium	Content

fixed

Short	Very very long content
Medium	Content

```
table {
  width: 100% ;
  border-collapse: separate ;
  border-spacing: 5px 20px ;
  table-layout: auto ;
}
th, td {
  border: 1px solid black ;
}
```

```
table {
  width: 100% ;
  border-collapse: separate ;
  border-spacing: 5px 20px ;
  table-layout: fixed ;
}
th, td {
  border: 1px solid black ;
}
```

[\[Link\]](#)

A clean slate

- Browsers use built-in **user agent style sheets** to render HTML elements with default styles, which can vary across browsers, **causing inconsistencies**.
- Even with a custom style sheet, element may still inherit these defaults, **leading to unexpected results**.
- There are two methods for getting a consistent starting point for applying your own styles
 - CSS reset (old approach)
 - normalize.css

CSS Reset

- A [CSS reset](#) is a collection of styles that overrides all user agent styles (browser defaults) and give your [neutral](#) starting point for your own styles.
- The most popular reset was written by Eric Meyer that removes margins, padding and borders and sets the font to 100% for all elements:

```
/* http://meyerweb.com/eric/tools/css/reset/  
   v2.0 | 20110126 License: none (public domain)*/  
html, body, div, span, applet, object, iframe,  
h1, h2, h3, h4, h5, h6, p, blockquote, pre,  
a, abbr, /*... MORE ELEMENTS...*/ section, summary,  
time, mark, audio, video {  
    margin: 0;  
    padding: 0;  
    border: 0;  
    font-size: 100%;  
    font: inherit;  
    vertical-align: baseline;  
}  
/* Styles continue... */
```

Normalize.css

- [Normalize.css](#) is a reset style sheet that retain some initial browser styles but tweaks them for consistency across browsers. It prevents needing to write style for every element and provides a reasonable starting point.

CSS reset

List Item
List Item
List Item
List Item
List Item
List Item
Big letters
Less big of letters
Getting smaller
Hello world
Kid Rock
Ant man
Lorem ipsum dolor sit amet.
Lorem ipsum dolor sit amet, consectetur adipisicing elit. Fugit dignissimos iusto maxime, quibusdam ut harum
consectetur 1 rem consequuntur sunt vero!
Lorem ipsum dolor sit amet, consectetur adipisicing elit. Aperiam iste, soluta quibusdam ullam error qui atque 7,
expedita quod dicta vel laudantium tenetur, quos, voluptatibus aspernatur enim magni velit. Distinctio, adipisci.

[Singles in your area](#)
Don't click fancy
loud
loud?
copyright forever

Normalize.css

- List Item
- List Item
- List Item
- List Item
- List Item

Big letters

Less big of letters

Getting smaller

Hello world

Kid Rock

Ant man

Lorem ipsum dolor sit amet.

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Fugit dignissimos iusto maxime, quibusdam ut harum
consectetur 1 rem consequuntur sunt vero!

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Aperiam iste, soluta quibusdam ullam error qui atque 7,
expedita quod dicta vel laudantium tenetur, quos, voluptatibus aspernatur enim magni velit. Distinctio, adipisci.

[Singles in your area](#)

Don't click fancy

loud

loud?

copyright forever

Image Replacement Techniques (IRT)

- Before web fonts, designers used images for custom text styles beyond basic fonts like Times or Helvetica. While less common now, **CSS-based image replacement** is still useful for elements like stylized logos.
- The **image replacement technique** (IRT) keeps the text in the markup for accessibility, using a background image while hiding the text visually, ensuring it remains available for search engines and screen readers.

Image Replacement Technique

The markup:

```
<h1 id="logo">Jenware</h1>
```

The style rule:

```
#logo {  
  width: 450px;  
  height: 80px;  
  background: url(jenware.png) no-repeat;  
  text-indent: -9999px;  
}
```

What users see:

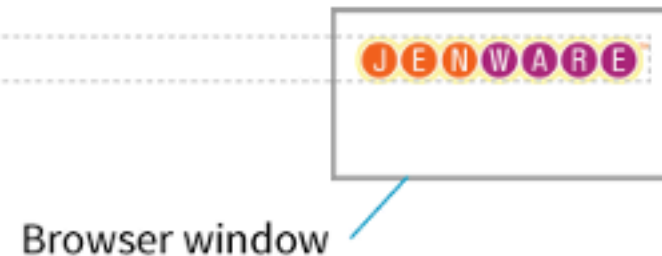


What is actually happening:

text-indent: -9999px;

Jenware

The h1 text content is pushed way off to the left, outside the browser window.



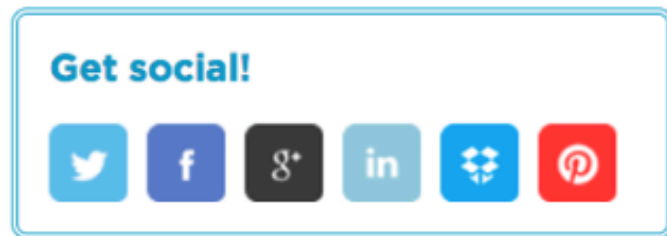
IRT Considerations

- The downside to any image replacement approach is that it requires an extra request to the server for every image used.
- Before choosing an image replacement, consider whether a web font or inline image with alt text can be do the job.

CSS Sprites

- To improve site performance, we should reduce the number of requests the page makes to the servers.
- One strategy for reducing the number of image requests is to combine all little images into a big image file so that only one image gets requested.
- The large image that contains multiple images is known as a [sprite](#).
- That image gets positioned in the element via the [background-position](#) property.
 - This way that only relevant portion of the image is visible.

CSS Sprites

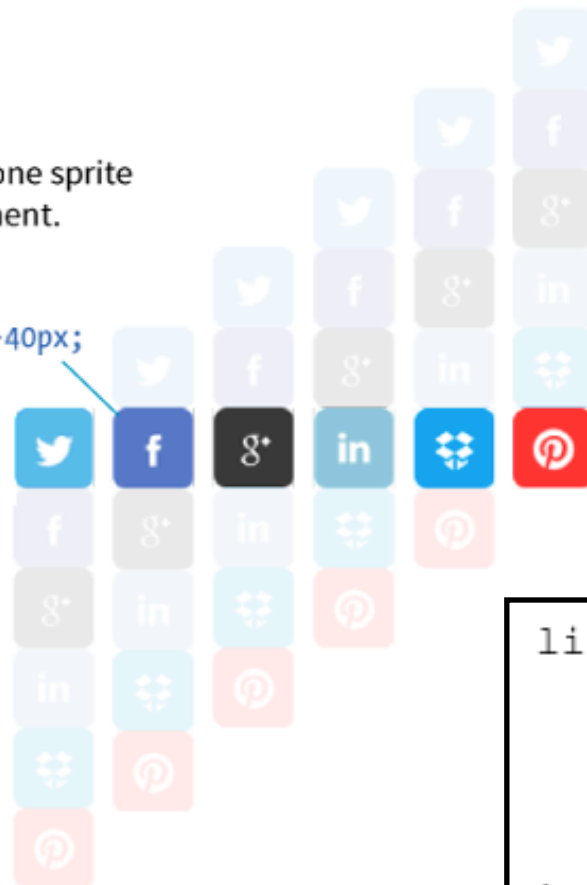


The separate icons in this panel are contained on one sprite image (*social.png*) that is positioned in each `a` element.

social.png



`background-position: 0, -40px;`
`background-position: 0, 0;`



THE MARKUP

```
<ul>
  <li><a href="" class="hide twitter">Twitter</a></li>
  <li><a href="" class="hide fb">Facebook</a></li>
  <li><a href="" class="hide gplus">Google+</a></li>
  <li><a href="" class="hide linkedin">LinkedIn</a></li>
  <li><a href="" class="hide dropbox">Dropbox</a></li>
  <li><a href="" class="hide pinterest">Pinterest</a></li>
</ul>
```

THE STYLES

```
.hide {
  text-indent: 100%;
  white-space: nowrap;
  overflow: hidden;
}
```

```
li a {
  display: block;
  width: 40px;
  height: 40px;
  background-image: url(social.png);
}
li a.twitter { background-position: 0 0; }
li a.fb { background-position: 0 -40px; }
li a.gplus { background-position: 0 -80px; }
li a.linkedin { background-position: 0 -120px; }
li a.dropbox { background-position: 0 -160px; }
li a.pinterest { background-position: 0 -200px; }
```


CSS Feature Detection

- Useful new CSS properties emerge regularly, but it takes a while for new CSS features to be supported in all browsers.
- Feature detection let you to test for support of a particular feature and provide appropriate fallbacks for non-supporting browsers.
- Two common detection methods:
 - CSS feature queries (@supports rule)
 - Modernizr (JavaScript library)
- In the past, styles were delivered based on the browser version. Testing for individual features is a better approach.

CSS Feature Queries (@supports)

- `@supports` rule tests for support of a particular property and value pair.
- If the browser passes, the rule inside the brackets are applied:

```
@supports [not | and | or ] (property: value) {
```

```
    /* Style rules for supporting browsers here */
```

```
}
```

- **TIP:** Provide fallback styles for non-supporting browsers first and then override them with the preferred rules.

CSS Feature Queries



Original image (no effect)



As seen on browsers that support
`mix-blend-mode: multiply;`



Fallback for non-supporting browsers
(`opacity: .5`)

The `mix-blend-mode` property is specified for browsers that support it. Other browsers get a similar effect with the widely supported `opacity` property.

```
#container {  
  background-color: #96D4E7;  
  padding: 5em;  
}  
.blend img {  
  opacity: .5;  
}  
@supports (mix-blend-mode: multiply) {  
  .blend img {  
    mix-blend-mode: multiply;  
    opacity: 1;  
  }  
}
```

CSS Feature Queries

- PROS:

- Doesn't rely on JavaScript
- Can take advantage of cutting-edge properties safely

- CONS

- Limited browser support
- **CAUTION:** Some browser that don't support features queries **do** support newer properties you might test for (for example, Flexbox). The new properties won't apply if they are in an unsupported **@supports** rule.

Modernizr

- A JavaScript library that tests for HTML5 and CSS features.
- It indicates the result (support or no support) with a class name applied to the html element (and in a JavaScript object for scripting):

```
<html class="js flexbox">  
<html class="js no-flexbox">
```

- Use the generated class name in the selector to separate styles for supporting and non-supporting browsers:

```
.flexbox nav {  
    /* flexbox styles for the nav element here */  
}  
.no-flexbox nav {  
    /* fallback styles for the nav element here */  
}
```

Modernizr

1. Go to modernizr.com to customize, build and download the Modernizer.js script.
2. Put the [Modernizer.js](#) file in the same directory as the file for your site.
3. Add the script to the head of the HTML document, before linked style sheets or other scripts.
4. Add class="no-js" to the [html](#) element (it get overwritten if there is JavaScript):

```
<html class="no-js">  
<head>  
  <script src="modernizr-custom.js"></script>  
  <!--other scripts and style sheets -->  
</head>
```

5. [Modernizr](#) will change it to [js](#) one it detects that the browser supports JavaScript. If JavaScript (and therefore [Modernizr](#)) fails to run, you will not know whether or not features are supported.

Modernizr

- PROS:
 - Easy to use, with thorough documentation
 - Excellent browser support
- CONS:
 - Relies on JavaScript (which may not be enabled)
 - Slightly slow than CSS feature queries